

TRABAJO PRÁCTICO N.º 1 — Food Store: Del dominio al esquema

Modelo ER, Modelo Relacional, Normalización 3FN/BCNF y DDL

Materia: Base de Datos I — Tecnicatura Universitaria en Programación a Distancia (UTN)

Proyecto integrador: Food Store — Semana 1

Alumno: Apellido_Nombre — Legajo: [completar] — Fecha: 2026-08-27

Archivos asociados: docs/er.dbml (Crow's Foot, dbdiagram.io), schema.sql

Parte 1 — Modelo Entidad-Relación (25%)

Diagrama ER

- **Fuente:** docs/er.dbml — pegar en <https://dbdiagram.io> y exportar a docs/er.png y docs/er.pdf .
- **Notación:** Crow's Foot
- **Entidades:** categoria , producto , cliente , pedido + asociativa pedido_producto .
- **Cardinalidades:**
 - categoria (1) — (N) producto — R1: todo producto pertenece exactamente a una categoría; categoría puede tener 0...N productos.
 - cliente (1) — (N) pedido — R2: todo pedido pertenece exactamente a un cliente; cliente puede tener 0...N pedidos.
 - pedido (N) — (M) producto vía pedido_producto — R3/R4: incluye atributos cantidad , precio_unitario .
- **Participación:**
 - producto **total** respecto a categoria (obligatoria, categoria_id NOT NULL), categoria **parcial** (opcional).
 - pedido **total** respecto a cliente (obligatoria), cliente **parcial**.
 - pedido_producto ambas FK **NOT NULL** por ser PK compuesta; a nivel instancia, pedido puede existir sin filas en intermedia si se permite borrador (parcial), o exigir al menos 1 (total) — ver P2-Q2.

Diccionario de entidades y atributos

Entidad	Atributo	Tipo conceptual	Clave	Nulo	Dominio / Restricción	Descrip
categoria	id	numérico	PK	NOT NULL	GENERATED ALWAYS AS IDENTITY	Identific surrogat autogen
	nombre	texto	Cand. UNIQUE	NOT NULL	VARCHAR(80) UNIQUE	Nombre categorí (Pizzas, Bebidas Único p evitar duplicac
	activo	booleano	—	NOT NULL	DEFAULT TRUE	R7: baja lógica, conserv. historial
	created_at	fecha/hora	—	NOT NULL	DEFAULT now()	Auditorí creació

producto	id	numérico	PK	NOT NULL	IDENTITY	Identific surrogate
	nombre	texto	—	NOT NULL	VARCHAR(100)	Nombre product (Muzzar
	precio	numérico	—	NOT NULL	NUMERIC(10,2) CHECK >=0	R5: prec actual; r negativo
	stock	entero	—	NOT NULL	INTEGER CHECK >=0	R5: stock disponit negativ
	activo	booleano	—	NOT NULL	DEFAULT TRUE	R7: baja lógica.
	categoria_id	numérico	FK categoria.id	NOT NULL	REFERENCES	R1: pertenece obligato categorí
	created_at	fecha/hora	—	NOT NULL	DEFAULT now()	Auditorí
cliente	id	numérico	PK	NOT NULL	IDENTITY	Identific surrogate
	nombre	texto	—	NOT NULL	VARCHAR(100)	Nombre complet (compue ver Q3).
	email	texto	Cand. UNIQUE	NOT NULL	VARCHAR(255) UNIQUE	R6: iden unívoca al cliente
	created_at	fecha/hora	—	NOT NULL	DEFAULT now()	Auditorí
pedido	id	numérico	PK	NOT NULL	IDENTITY	N.º de p
	fecha	fecha/hora	—	NOT NULL	TIMESTAMPZ DEFAULT now()	Fecha/h del pedi
	forma_pago	enumerado	—	NOT NULL	ENUM('EFECTIVO','TARJETA','TRANSFERENCIA')	Dominic cerrado.
	cliente_id	numérico	FK cliente.id	NOT NULL	REFERENCES	R2: pedi pertenece un client
	created_at	fecha/hora	—	NOT NULL	DEFAULT now()	Auditorí
pedido_producto	pedido_id	numérico	PK, FK pedido.id	NOT NULL	REFERENCES	Parte de compue R3.
	producto_id	numérico	PK, FK producto.id	NOT NULL	REFERENCES	Parte de compue R3.
	cantidad	entero	—	NOT NULL	CHECK >0	R4: unid pedidas

	precio_unitario	numérico	—	NOT NULL	NUMERIC(10,2) CHECK >=0	R4: precio congelado histórico
--	-----------------	----------	---	-------------	-------------------------	--------------------------------

Decisión PK intermedia: PK compuesta (pedido_id, producto_id) — elegida por consigna. Garantiza que un producto no se repita en más de una línea del mismo pedido (regla p4) y mantiene FKs NOT NULL por integridad de entidad. Alternativa surrogate id + UNIQUE(pedido_id, producto_id) descartada por simplicidad y fidelidad al modelo ER.

Preguntas guía — Parte 1

Q1. ¿Por qué la relación producto-pedido no puede ser 1:N y qué se perdería?

Es N:M por naturaleza: un pedido incluye varios productos distintos y un producto aparece en muchos pedidos. Forzar 1:N con pedido_id en producto limitaría cada producto del catálogo a venderse en un único pedido histórico; con producto_id en pedido limitaría cada pedido a un solo producto. Además, los atributos cantidad y precio_unitario son de la intersección (R4): si se ponen en producto se pierde trazabilidad histórica por pedido; si en pedido se fuerza misma cantidad/precio para todos los productos. Sin intermedia se viola R4 (precio histórico no inmutable).

Q2. ¿Qué entidad tiene participación parcial/total con categoría y qué pasa si se invierte?

- **Total:** producto — R1 exige que todo producto pertenezca exactamente a una categoría (categoria_id NOT NULL).
- **Parcial:** categoría — R1 permite categorías con 0 productos (recién creada).
- **Si se invierte:** producto parcial permitiría productos huérfanos sin categoría reportes/agrupaciones rotas. categoría total obligaría a que toda categoría tenga ≥1 producto siempre imposible crear categorías vacías para planificación.

Q3. ¿Atributo descomponible (ej. nombre completo nombre+apellido)?

cliente.nombre es descomponible en nombre + apellido. Separarlo facilita ordenar/buscar por apellido y personalización, pero aquí se deja **compuesto** como nombre único por (a) cumplir consigna de mínimo exigido (PDF: “si agregás atributos aclará”), (b) mantener 1:1 con la planilla de Parte 3 donde figura “Ana Gómez” en una columna, evitando complejidad innecesaria en el prototipo. Decisión documentada y reversible.

Parte 2 — Derivación al Modelo Relacional (20%)

Listado completo (notación tabla (atributos) — PK subrayada, FK con)

1. categoria (<u>id</u> PK, nombre UNIQUE NOT NULL, activo NOT NULL DEFAULT TRUE, created_at NOT NULL DEFAULT now())
2. producto (<u>id</u> PK, nombre NOT NULL, precio NOT NULL CHECK >=0, stock NOT NULL CHECK >=0, activo NOT NULL DEFAULT TRUE, created_at NOT NULL DEFAULT now(), categoria_id NOT NULL categoria.id)
3. cliente (<u>id</u> PK, nombre NOT NULL, email NOT NULL UNIQUE, created_at NOT NULL DEFAULT now())
4. pedido (<u>id</u> PK, fecha NOT NULL DEFAULT now(), forma_pago NOT NULL ENUM, cliente_id NOT NULL cliente.id, created_at NOT NULL DEFAULT now())
5. pedido_producto (<u>pedido_id</u> pedido.id, <u>producto_id</u> producto.id, cantidad NOT NULL CHECK >0, precio_unitario NOT NULL CHECK >=0) — **PK(pedido_id, producto_id)**

Reglas aplicadas:

- Entidad fuerte tabla con su PK.
- 1:N FK del lado 1 en tabla N (no tabla nueva) — aplicado a categoria producto y cliente pedido.
- N:M tabla intermedia con FKs a ambas entidades + atributos propios — aplicado a pedido_producto.
- PK intermedia: compuesta por decisión fijada; justificada arriba.

Preguntas guía — Parte 2

Q1. ¿Qué pasa si la intermedia no tiene ambas FK como NOT NULL?

1. **Viola integridad de entidad:** al ser PK compuesta, ningún componente puede ser NULL por definición relacional filas no identificables.

2. **Viola integridad referencial y semántica:** pedido_id NULL línea huérfana sin pedido facturable; producto_id NULL línea sin producto contable/stock. Se pierde trazabilidad R3/R4 y se permiten datos inconsistentes.

Q2. ¿Si el pedido pudiera existir sin productos (borrador), cambia la participación con la intermedia?

Sí, conceptualmente: en Parte 1 se modeló pedido total respecto a pedido_producto (pedido cerrado debe tener ≥ 1 línea). Si se permite borrador vacío, pedido pasa a parcial (puede existir sin filas en intermedia). **Impacto físico:** nulo — las FKs de pedido_producto siguen NOT NULL y PK compuesta; solo se permite INSERT INTO pedido sin INSERT en intermedia. A nivel aplicativo facilita flujo en dos pasos (alta cabecera luego líneas) dentro de una transacción; si se exigiera totalidad dura habría que insertar cabecera+líneas atómicamente o usar triggers/assertions más complejos.

\newpage

Parte 3 — Normalización hasta 3FN/BCNF (25%)

1. Relación universal de partida (planilla plana)

Extracto PDF p4 (7 filas, una por línea de producto vendido dentro de un pedido):

N.º pedido	Fecha	Cliente	Producto	Categoría	Precio unitario	Cant.	Subtotal	Forma de pago
1	01/03/2026	Ana Gómez	Muzzarella	Pizzas	1000.00	2	2000.00	EFFECTIVO
1	01/03/2026	Ana Gómez	Coca 1.5L	Bebidas	800.00	1	800.00	EFFECTIVO
2	01/03/2026	Luis Paz	Napolitana	Pizzas	1500.00	1	1500.00	TARJETA
3	05/03/2026	Ana Gómez	Muzzarella	Pizzas	1050.00	3	3150.00	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	Coca 1.5L	Bebidas	800.00	4	3200.00	EFFECTIVO
4	06/03/2026	Marta Ruiz	Napolitana	Pizzas	1500.00	2	3000.00	EFFECTIVO
5	07/03/2026	Luis Paz	Muzzarella	Pizzas	1050.00	1	1050.00	TARJETA

Esquema universal U :

U(nro_pedido, fecha, cliente, producto, categoria, precio_unitario, cantidad, subtotal, forma_pago)

Reglas de negocio para FDs (PDF p4):

- Cada pedido tiene única fecha, único cliente y única forma de pago.
- Cada producto pertenece a única categoría.
- Dentro de un mismo pedido, un producto no se repite en más de una línea.
- El precio de un producto puede variar en el tiempo (Muzzarella 1000 en pedido 1 1050 en pedido 3).

Supuesto de la consigna: cliente (nombre) y producto (nombre) identifican unívocamente en esta muestra (en BD real serán IDs numéricos).

2. Desarrollo paso a paso (7 pasos)

Paso 1 — Clave(s) candidata(s) de U

Razonamiento: ¿Qué combinación mínima identifica una fila de forma única?

- nro_pedido solo no identifica: pedido 1 tiene 2 filas, pedido 4 tiene 2 filas.
- producto solo no identifica: Muzzarella aparece en pedidos 1, 3 y 5; Coca 1.5L en 1 y 4.
- cliente no identifica, fecha repetida, categoria repetida.

La consigna aclara: “dentro de un mismo pedido, un producto no se repite en más de una línea”. Por lo tanto, la combinación (nro_pedido, producto) es única por fila.

Clave candidata principal: PK = (nro_pedido, producto) — clave compuesta mínima.

Paso 2 — Dependencias funcionales (FDs)

Listado exhaustivo apoyado en datos y reglas:

1. nro_pedido fecha, cliente, forma_pago
2. producto categoria
3. (nro_pedido, producto) cantidad, precio_unitario, subtotal
4. precio_unitario, cantidad subtotal

FD clave / pista del PDF — ¿de qué depende precio_unitario ?

Hipótesis producto precio_unitario es **falsa**. Contraejemplo: Muzzarella 1000 en fila 1 pero Muzzarella 1050 en filas 3 y 5. El precio cambia en el tiempo. Por lo tanto, (nro_pedido, producto) precio_unitario . Esto preserva R4.

Notación formal:

```
F = {  
  nro_pedido fecha, cliente, forma_pago,  
  producto categoria,  
  (nro_pedido, producto) cantidad, precio_unitario, subtotal,  
  (cantidad, precio_unitario) subtotal  
}
```

Paso 3 — Verificación 1FN

Todos los atributos atómicos, sin grupos repetidos. **Conclusión:** U ya está en 1FN. No requiere corrección.

Paso 4 — Verificación 2FN (eliminar dependencias parciales)

Clave compuesta: (nro_pedido, producto) .

- nro_pedido fecha, cliente, forma_pago — depende solo de parte de la clave **viola 2FN**.
- producto categoria — depende solo de parte de la clave **viola 2FN**.
- (nro_pedido, producto) cantidad, precio_unitario, subtotal — depende de clave completa cumple.

Descomposición 2FN:

- R1: pedido(nro_pedido PK, fecha, cliente, forma_pago)
- R2: producto_categoria(producto PK, categoria)
- R3: linea_pedido(nro_pedido, producto PK, cantidad, precio_unitario, subtotal) — PK(nro_pedido, producto)

Paso 5 — Verificación 3FN (eliminar dependencias transitivas)

- pedido(nro_pedido, fecha, cliente, forma_pago) — sin transitiva cumple 3FN.
- producto_categoria(producto, categoria) — directa cumple 3FN.
- linea_pedido(nro_pedido, producto, cantidad, precio_unitario, subtotal) — (nro, prod) (cantidad, precio) subtotal transitiva por subtotal derivado.

Corrección 3FN: separar categoria(categoria PK) y producto(producto PK, categoria FK) , y **eliminar subtotal** del modelo normalizado (o tratarlo como columna generada).

Tablas tras 3FN:

```
pedido(nro_pedido PK, fecha, cliente, forma_pago)  
categoria(categoria PK)  
producto(producto PK, categoria FK categoria)  
linea_pedido(nro_pedido FK, producto FK, cantidad, precio_unitario) — PK(nro_pedido, producto)
```

Paso 6 — Verificación BCNF

Para toda FD no trivial $X \rightarrow Y$, X es superclave.

- pedido(nro_pedido fecha, cliente, forma_pago) — PK cumple BCNF.
- categoria — sin FD cumple BCNF.
- producto(producto categoria) — PK cumple BCNF.
- linea_pedido((nro_pedido, producto) cantidad, precio_unitario) — PK compuesta cumple BCNF.

Si se mantuviera subtotal : cantidad, precio_unitario subtotal con determinante no superclave violaría BCNF. Por eso se excluye.

Conclusión: con subtotal excluido, todas cumplen BCNF. No hay excepciones.

Paso 7 — Conjunto final de tablas normalizadas (3FN/BCNF)

- categoria(*categoria) — PK: categoria
- producto(*producto, categoria FK categoria) — PK: producto
- cliente(*cliente) — PK: cliente
- pedido(*nro_pedido, fecha, cliente FK cliente, forma_pago) — PK: nro_pedido
- linea_pedido(*nro_pedido FK pedido, *producto FK producto, cantidad, precio_unitario) — PK: (nro_pedido, producto)

Equivalencia con ER: con IDs numéricos surrogate, cliente cliente.id , producto producto.id , isomorfo a categoria , producto , cliente , pedido , pedido_producto .

```
categoria(*categoria)
producto(*producto, categoria categoria)
cliente(*cliente)
pedido(*nro_pedido, fecha, forma_pago, cliente cliente)
linea_pedido(*nro_pedido pedido, *producto producto, cantidad, precio_unitario)
```

3. Preguntas de integración

Q1. Compará tablas normalizadas con ER (Parte 2). ¿A qué entidad corresponde cada una? ¿Atributos ER ausentes en planilla? ¿Qué dice sobre modelar vs normalizar?

Tabla normalizada	Entidad ER	Observaciones
categoria	categoria	Misma
producto	producto	Parcial: falta precio lista, stock , activo (R5/R7)
cliente	cliente	Parcial: falta email UNIQUE (R6)
pedido	pedido	Corresponde: nro_pedido id , fecha , forma_pago , cliente
linea_pedido	pedido_producto	Misma N:M con cantidad , precio_unitario

Atributos ER ausentes en planilla: cliente.email , producto.precio , producto.stock , producto.activo , categoria.activo , created_at . **Conclusión:** ER es prospectivo/normativo (R1-R7), normalización es retrospectiva/descriptiva (solo lo registrado). Se concilian en DDL Parte 4: ER aporta estructura normativa, normalización valida histórico sin redundancia y revela clave temporal de precio_unitario .

Q2. subtotal calculable cantidad*precio_unitario ¿Conviene almacenar o recalcular? ¿Viola FN?

subtotal derivado: almacenarlo como base genera (nro,prod) (cantidad,precio) subtotal transitiva y cantidad,precio subtotal no BCNF viola 3FN/BCNF si es atributo base. En modelo normalizado se excluye. En DDL físico es válido como GENERATED ALWAYS AS (cantidad*precio_unitario) STORED o vista; si se almacena sin CHECK/columna generada, hay riesgo de inconsistencia. Es decisión de rendimiento/trazabilidad, no de normalización pura.

Parte 4 — Referencia DDL

El DDL completo está en schema.sql:1 y concilia ER y normalización. Ver ese archivo para tipos (BIGINT GENERATED ALWAYS AS IDENTITY , NUMERIC(10,2) , ENUM), PK , FK ON DELETE RESTRICT , UNIQUE(email) , 3 CHECK , NOT NULL , DEFAULT , 2 INDEX comentados.

Anexo — Declaración de Uso de IA (DUIA)

Ver `DUIA.md:1` — OpenCode + Muse Spark como motor primario (Sección 7 PDF), prompts y verificación humana por parte.